PRADITA
University

IF140303-Web Application Development

Session-05:

Avatar Generator 1/2

PRU/SPMI/FR-BM-18/0222

Alfa Yohannis



Contents

PRADITA University

1. Introduction
   - Avatar Generator
   - Avatar Pipeline
   - Avatar Computation
2. Code Skeleton
3. Image Structure Definition
4. Module Structure
5. The Avatar Image Structure
6. Avatar Generation Module
7. Avatar Generation Function
8. Compute the Hash
9. Resizing Hash List
10. Selecting Avatar Color
11. Creating the Avatar Grid
12. Reflecting Row Symmetry
13. Starting the Application
14. Running the Application
15. Conclusion

Introduction

PRADITA University

This section explains the development of an **avatar generator** built using the Elixir programming language. The program showcases key Elixir features:

- `defstruct` for defining structured data types,
- `defmodule` for modular organization,
- `Application` behavior for managing runtime lifecycle.

The generator takes a text input (e.g., a username) and computes its **MD5 hash**, which is a **128-bit** (**16-byte**) fixed-length value. **Each byte** is **converted** into an **integer** between **0 and 255**, forming a sequence of numbers that represents the hash in numeric form. These integers are then used to:

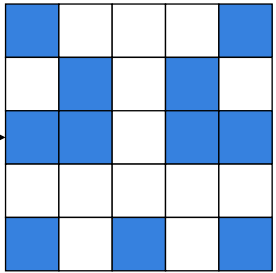
- determine which grid cells will be filled,
- assign colors or symmetry patterns to each region.

Thus, every unique input string produces a distinct yet deterministic avatar.

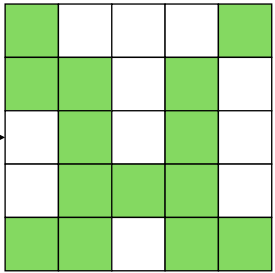
Avatar Generator

PRADITA University

"wolverine" →



"superman" →



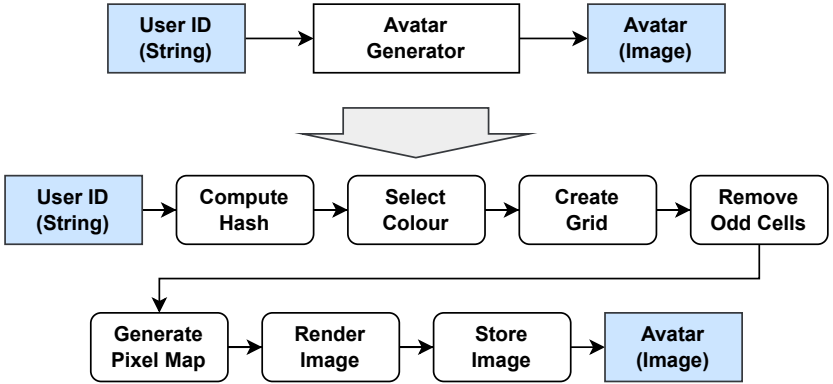
wolverine's md5 hash = [54, 129, 223, 141, 4, 71, 14, 204, 101, 5, 59, 121, 14, 25, 160, 101]

superman's md5 hash = [132, 217, 97, 86, 138, 101, 7, 58, 59, 207, 14, 178, 22, 178, 165, 118]

The 128-bit MD5 hash is split into 16 integers (0–255), each encoding color or grid placement. These values define the avatar's pattern and symmetry, ensuring every input yields a unique, consistent design.

Avatar Pipeline

PRADITA University



Input text is hashed (MD5, 128-bit/16 bytes) and drives color and layout. Functions run sequentially to transform data into a rendered PNG image.

Pipeline Code

PRADITA University

```
compute_hash
|> select_color
|> create_grid
|> remove_odd_cells
|> generate_pixel_map
|> render_image
|> store_image
```

compute_hash: MD5 16 integers (0–255). **select_color**: derives RGB from first bytes. **create_grid**: builds 5×5 layout with symmetry. **remove_odd_cells**: keep even-valued cells. **generate_pixel_map**: map cells to coordinates. **render_image**/**store_image**: draw and save PNG.

Avatar Computation

PRADITA University

- The avatar is generated as a **5×5 grid** image.
- **Columns 4 and 5 mirror columns 2 and 1**, creating symmetry and visual balance.
- **The first three columns** use integer values of the MD5 hash of the `username`.
- **Even** values fill colored cells; **odd** values remain white.
- **The first three hash bytes** define the RGB color used for the avatar.

0 **54**	1 **129**	2 **223**	3 **129**	4 **54**
5 **141**	6 **4**	7 **71**	8 **4**	9 **141**
10 **14**	11 **204**	12 **101**	13 **204**	14 **14**
15 **5**	16 **59**	17 **121**	18 **59**	19 **5**
20 **14**	21 **25**	22 **160**	23 **25**	24 **14**

The first 3 values are the RGB colour

[54, 129, 223, 141, 4, 71, 14, 204, 101, 5, 59, 121, 14, 25, 160, 101]

Figure: Hash integers, mirroring, and RGB mapping.

Avatar Generator Code Skeleton

PRADITA University

This code defines the main structure of the `AvatarGenerator` module in Elixir. Each function represents a stage in the avatar creation process — from hashing input data to generating and storing the final image. The modular design makes the pipeline readable, maintainable, and easy to extend. Keseluruhan Kode `src/avatar.ex`.

```elixir
defmodule AvatarGenerator do
  use Application

  def generate(input) do
  end

  def compute_hash(input) do
  end

  def select_color(%Avatar.Image{
    hash: [r, g, b | _tail]} = image)
        do
  end

  def create_grid(%Avatar.Image{
    hash: hash} = image) do
  end

  def reflect_row(row) do
  end

  def resize_list(list) do
  end

  def start(_type, _args) do
  end
end
```

Image Structure Definition

PRADITA University

- The `Avatar.Image` module defines a simple data structure used throughout the avatar generation process.
- It stores the computed MD5 hash and the selected RGB color, serving as a container for passing data between functions.
- The keyword `defstruct` in Elixir defines a structure (similar to a record or lightweight object) with named fields and default values.
- This enables the creation of consistent, easily managed data instances that can be passed and modified within the program.

Listing 1: Definition of src/image.ex

```
1 defmodule Avatar.Image do
2   defstruct hash: nil, color: nil
3 end
```

Module Structure

PRADITA University

The program is organized into two main modules, each serving a specific role in the avatar generation process:

- **Avatar.Image**: Defines a structured data type (`defstruct`) to store avatar-related information, including the MD5 hash and the RGB color used for rendering.
- **AvatarGenerator**: Serves as the core logic module responsible for computing the hash, selecting colors, generating the symmetric grid, and producing the final avatar image.

Together, these modules separate data representation from processing logic, making the program modular, reusable, and easier to maintain.

Defining the Avatar Image Structure

PRADITA University

- The `Avatar.Image` module defines a structure to store avatar-related data.
- It holds two main attributes:
  - `hash` — stores the MD5 hash generated from the input string.
  - `color` — stores the RGB color derived from the hash values.
- The `defstruct` keyword defines these fields with default values of `nil`.
- This structure ensures consistent data handling during the avatar generation process.

Listing 2: Defining the Avatar Image structure in `lib/image.ex`

```
1 defmodule Avatar.Image do
2   defstruct hash: nil,
        color: nil
3 end
```

Avatar Generator Module

PRADITA University

- The `AvatarGenerator` module acts as the main logic controller for creating avatars.
- It uses the `Application` behavior, which enables defining the `start/2` function to initialize the app.
- The module coordinates the full workflow:
  - computing hashes,
  - selecting colors,
  - generating grids,
  - rendering and saving the final image.
- It serves as the core engine combining helper functions into a structured, functional pipeline.

Listing 3: Definition of the Avatar Generator module in `lib/avatar.ex`

```
1 defmodule AvatarGenerator do
2   use Application
```

Avatar Generation Function

PRADITA University

- The `generate/1` function is the main entry point for creating an avatar.
- It accepts a string `input` (e.g., a username) and performs:
  - hash computation using MD5,
  - color selection based on hash values,
  - grid creation for avatar visualization.
- The Elixir pipeline operator (`|>`) passes results between functions, ensuring a clear and functional data flow.

Listing 4: Main function for generating avatars

```elixir
def generate(input) do
  input
  |> compute_hash
  |> select_color
  |> create_grid
end
```

Computing the Hash

PRADITA University

- The `compute_hash/1` function generates an MD5 hash from the given input string.
- It uses the built-in `:crypto` module to produce a 128-bit binary hash.
- The hash is then converted into a list of integers (0–255) using `:binary.bin_to_list/1`.
- The `resize_list/1` helper ensures the list length is a multiple of three, aligning it with the RGB grouping used later.
- The resulting hash list is stored in a new `%Avatar.Image{}` struct.

Listing 5: Computing the hash from the input string

```elixir
def compute_hash(input) do
  hash =
    :crypto.hash(:md5, input)
    |> :binary.bin_to_list()
    |> resize_list

  IO.inspect(hash)
  %Avatar.Image{hash: hash}
end
```

Helper Function: Resizing the Hash List

- The `resize_list/1` function ensures the hash list length is a multiple of three for proper RGB grouping and grid layout.
- `length(list)` counts the elements, and `div(length(list), 3) * 3` calculates the nearest lower multiple of three.
- `Enum.take(list, full_chunks_count)` keeps only that portion, trimming excess elements for a balanced, symmetric avatar grid.

```elixir
def resize_list(list) do
  full_chunks_count = div(length(list), 3) * 3
  Enum.take(list, full_chunks_count)
end
```

Selecting the Avatar Color

PRADITA University

- The `select_color/1` function extracts the first three elements of the hash list.
- These values represent the red (`r`), green (`g`), and blue (`b`) components of the avatar's color.
- The color is stored as an RGB tuple `{r, g, b}` inside the `Avatar.Image` struct.
- This ensures that each input string produces a unique and consistent color scheme.

```elixir
def select_color(%Avatar.Image{hash: [r, g, b | _tail]} = image) do
  %Avatar.Image{image | color: {r, g, b}}
end
```

Creating the Avatar Grid

PRADITA University

- The `create_grid/1` function converts the hash list into a symmetric grid pattern.
- It splits the hash into rows of three elements using `Enum.chunk_every(3)`.
- Each row is mirrored with `reflect_row/1` to create horizontal symmetry.
- The resulting list of rows is flattened and indexed with `Enum.with_index/1`.
- This indexed grid defines the structure and position of each cell in the avatar.

```elixir
def create_grid(%Avatar.Image{hash: hash} = image) do
  hash
  |> Enum.chunk_every(3)
  |> Enum.map(&reflect_row/1)
  |> List.flatten
  |> Enum.with_index
end
```

Reflecting Rows for Symmetry

PRADITA University

- The helper function `reflect_row/1` ensures horizontal symmetry in the avatar grid.
- It takes a row of three elements and mirrors the first two values in reverse order.
- This produces a five-element row, creating a balanced, symmetric pattern.
- The mirrored layout contributes to the visual harmony of the generated avatar.

```elixir
def reflect_row(row) do
  [first, second | _tail] = row
  row ++ [second, first]
end
```

Starting the Application

PRADITA University

- The `start/2` function is automatically executed when the application launches.
- It calls `AvatarGenerator.generate("wolverine")` to create an avatar from the sample input.
- The resulting grid data structure is printed to the console using `IO.inspect/1`.
- The function returns `:ok, self()`, signaling successful startup and keeping the application process active.

```elixir
def start(_type, _args) do
  result = AvatarGenerator.generate("wolverine")
  IO.inspect(result)
  {:ok, self()}
end
```

Running the Application

PRADITA University

- To run the application, the configuration in `mix.exs` must specify the starting module.
- This is done by adding the `mod` attribute inside the `application` function.
- Here, `AvatarGenerator` is defined as the main module that initializes the app on startup.
- The `extra_applications` key includes default dependencies such as `:logger`.

```elixir
def application do
  [
    extra_applications: [:logger],
    mod: {AvatarGenerator, []}
  ]
end
```

Conclusion

PRADITA University

This chapter explained the implementation of an avatar generator using the Elixir programming language. The project utilizes:

- the `:crypto` module to compute MD5 hashes,
- Elixir's `struct` feature to store avatar data,
- and list operations to create a grid-based visual representation.

The application is managed through the `Application` behavior, enabling it to run as a standalone Elixir app. Overall, the design demonstrates modular, functional programming principles for generating unique, symmetric avatars.